



Chapter 3

Basics of Learning Theory

"Predicting the future isn't magic, it's artificial intelligence."

— Dave Waters

Learning is a process by which one can acquire knowledge and construct new ideas or concepts based on the experiences. Machine learning is an intelligent way of learning general concept from training examples without writing a program. There are many machine learning algorithms through which computers can intelligently learn from past data or experiences, identify patterns, and make predictions when new data is fed. This chapter deals with an introduction of concept learning of hypotheses space and modelling of machine learning.

Learning Objectives

- Understand the basics of learning theory and the key elements of machine learning
- Introduce Concept learning to obtain an abstraction and generalization from the data
- Learn about hypothesis representation language and to explore searching the hypothesis space using Find-S algorithm and its limitations
- Study about version spaces, List-Then-Eliminate algorithm and Candidate Elimination algorithm
- Introduce Inductive bias, a set of prior assumptions considered by a learning algorithm beyond the training data in order to perform induction
- Discuss the tradeoff between the two factors called bias and variance which exist when modelling a machine learning algorithm
- Understand the different challenges in model selection and model evaluation
- Study popular re-sampling model selection method called Cross-Validation (*K*-fold, LOOCV, etc.) to tune machine learning model
- Introduce the various learning frameworks and learn to evaluate hypothesis

3.1 INTRODUCTION TO LEARNING AND ITS TYPES

The process of acquiring knowledge and expertise through study, experience, or being taught is called as learning. Generally, humans learn in different ways. To make machines learn, we need to simulate the strategies of human learning in machines. But, will the computers learn? This question has been raised over many centuries by philosophers, mathematicians and logicians.

First let us address the question - What sort of tasks can the computers learn? This depends on the nature of problems that the computers can solve. There are two kinds of problems – well-posed and ill-posed. Computers can solve only well-posed problems, as these have well-defined specifications and have the following components inherent to it.

1. Class of learning tasks (T)
2. A measure of performance (P)
3. A source of experience (E)

The standard definition of learning proposed by Tom Mitchell is that a program can learn from E for the task T , and P improves with experience E . Let us formalize the concept of learning as follows:

Let x be the input and $\mathcal{X}$ be the input space, which is the set of all inputs, and Y is the output space, which is the set of all possible outputs, that is, yes/no.

Let D be the input dataset with examples, $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ for n inputs.

Let the unknown target function be $f: \mathcal{X} \rightarrow Y$, that maps the input space to output space. The objective of the learning program is to pick a function, $g: \mathcal{X} \rightarrow Y$ to approximate hypothesis f . All the possible formulae form a hypothesis space. In short, let H be the set of all formulae from which the learning algorithm chooses. The choice is good when the hypothesis g replicates f for all samples. This is shown in Figure 3.1.

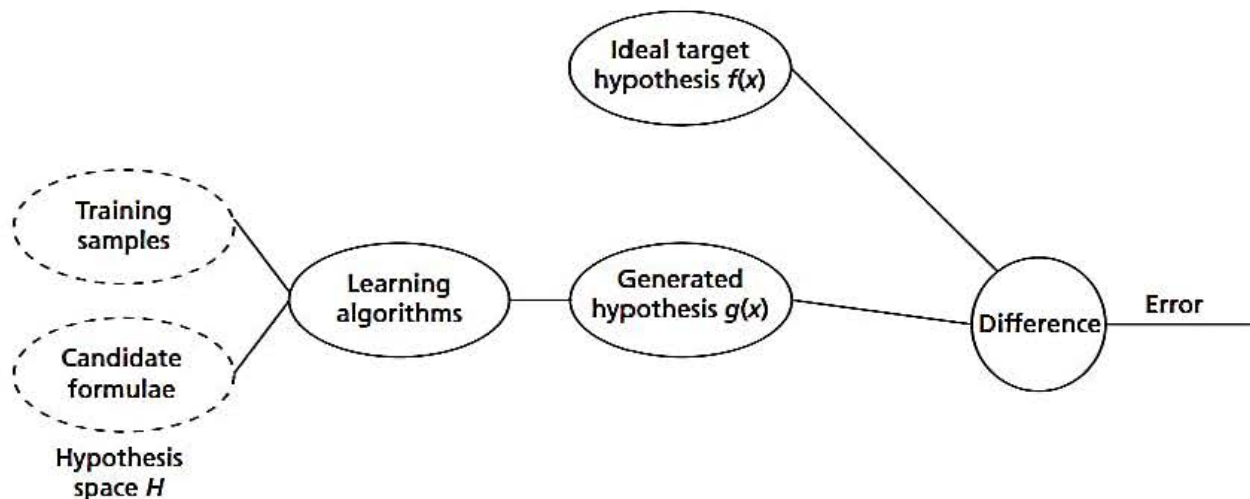


Figure 3.1: Learning Environment

It can be observed that training samples and target function are dependent on the given problem. The learning algorithm and hypothesis set are independent of the given problem. Thus, learning model is informally the hypothesis set and learning algorithm. Thus, learning model can be stated as follows:

$$\text{Learning Model} = \text{Hypothesis Set} + \text{Learning Algorithm}$$

Let us assume a problem of predicting a label for a given input data. Let D be the input dataset with both positive and negative examples. Let y be the output with class 0 or 1. The simple learning model can be given as:

$$\sum_{i=1}^D x_i w_i > \text{Threshold}, \text{ belongs to class 1 and}$$

$$\sum_{i=1}^D x_i w_i < \text{Threshold}, \text{ belongs to another class}$$

This can be put into a single equation as follows:

$$h(x) = \text{sign}\left(\left(\sum_{i=1}^D x_i w_i\right) + b\right)$$

where, $x_1, x_2, \dots, x_d$ are the components of the input vector, $w_1, w_2, \dots, w_d$ are the weights and +1 and -1 represent the class. This simple model is called perception model. One can simplify this by making $w_0 = b$ and fixing it as 1, then the model can further be simplified as:

$$h(x) = \text{sign}(w^T x).$$

This is called perception learning algorithm. The formal learning models are later discussed in Section 3.7 of this chapter.

Classical and Adaptive Machine Learning Systems

A classical machine learning system has components such as Input, Process and Output. The input values are taken from the environment directly. These values are processed and a hypothesis is generated as output model. This model is then used for making predictions. The predicted values are consumed by the environment.

In contrast to the classical systems, adaptive systems interact with the input for getting labelled data as direct inputs are not available. This process is called reinforcement learning. In reinforcement learning, a learning agent interacts with the environment and in return gets feedback. Based on the feedback, the learning agent generates input samples for learning, which are used for generating the learning model. Such learning agents are not static and change their behaviour according to the external signal received from the environment. The feedback is known as reward and learning here is the ability of the learning agent adapting to the environment based on the reward. These are the characteristics of an adaptive system.

Learning Types

There are different types of learning. Some of the different learning methods are as follows:

1. **Learn by memorization** or learn by repetition also called as *rote learning* is done by memorizing without understanding the logic or concept. Although rote learning is basically learning by repetition, in machine learning perspective, the learning occurs by simply comparing with the existing knowledge for the same input data and producing the output if present.
2. **Learn by examples** also called as **learn by experience** or previous knowledge acquired at some time, is like finding an *analogy*, which means performing *inductive learning* from observations that formulate a general concept. Here, the learner learns by inferring a general rule from the set of observations or examples. Therefore, inductive learning is also called as *discovery learning*.

3. **Learn by being taught** by an expert or a teacher, generally called as *passive learning*. However, there is a special kind of learning called *active learning* where the learner can interactively query a teacher/expert to label unlabelled data instances with the desired outputs.
4. **Learning by critical thinking**, also called as *deductive learning*, deduces new facts or conclusion from related known facts and information.
5. **Self learning**, also called as *reinforcement learning*, is a *self-directed learning* that normally learns from mistakes punishments and rewards.
6. **Learning to solve problems** is a type of *cognitive learning* where learning happens in the mind and is possible by devising a methodology to achieve a goal. Here, the learner initially is not aware of the solution or the way to achieve the goal but only knows the goal. The learning happens either directly from the initial state by following the steps to achieve the goal or indirectly by inferring the behaviour.
7. **Learning by generalizing explanations**, also called as *explanation-based learning (EBL)*, is another learning method that exploits domain knowledge from experts to improve the accuracy of learned concepts by supervised learning.

Acquiring general concept from specific instances of the training dataset is the main challenge of machine learning.

Scan for the figure showing 'Types of Learning'



3.2 INTRODUCTION TO COMPUTATION LEARNING THEORY

There are many questions that have been raised by mathematicians and logicians over the time taken by computers to learn. Some of the questions are as follows:

1. How can a learning system predict an unseen instance?
2. How close is the hypothesis h to f , when hypothesis f itself is unknown?
3. How many samples are required?
4. Can we measure the performance of a learning system?
5. Is the solution obtained local or global?

These questions are the basis of a field called 'Computational Learning Theory' or in short (COLT). It is a specialized field of study of machine learning. COLT deals with formal methods used for learning systems. It deals with frameworks for quantifying learning tasks and learning algorithms. It provides a fundamental basis for study of machine learning. It deals with Probably Approximate Learning (PAC) and Vapnik-Chervonenkis (VC) dimensions. The focus of PAC is the quantification of the computational difficulty of learning tasks and algorithms and the computation capacity quantification is the focus of VC dimension.

Computational Learning Theory uses many concepts from diverse areas such as Theoretical Computer Science, Artificial Intelligence and Statistics. The core concept of COLT is the concept of

learning framework. One such important framework is PAC. The learning framework is discussed in a detailed manner in Section 3.7. COLT focuses on supervised learning tasks. Since the complexity of analyzing is difficult, normally, binary classification tasks are considered for analysis.

3.3 DESIGN OF A LEARNING SYSTEM

A system that is built around a learning algorithm is called a learning system. The design of systems focuses on these steps:

1. Choosing a training experience
2. Choosing a target function
3. Representation of a target function
4. Function approximation

Training Experience

Let us consider designing of a chess game. In direct experience, individual board states and correct moves of the chess game are given directly. In indirect system, the move sequences and results are only given. The training experience also depends on the presence of a supervisor who can label all valid moves for a board state. In the absence of a supervisor, the game agent plays against itself and learns the good moves, if the training samples cover all scenarios, or in other words, distributed enough for performance computation. If the training samples and testing samples have the same distribution, the results would be good.

Determine the Target Function

The next step is the determination of a target function. In this step, the type of knowledge that needs to be learnt is determined. In direct experience, a board move is selected and is determined whether it is a good move or not against all other moves. If it is the best move, then it is chosen as: $B \rightarrow M$, where, B and M are legal moves. In indirect experience, all legal moves are accepted and a score is generated for each. The move with largest score is then chosen and executed.

Determine the Target Function Representation

The representation of knowledge may be a table, collection of rules or a neural network. The linear combination of these factors can be coined as:

$$V = w_0 + w_1x_1 + w_2x_2 + w_3x_3$$

where, x_1 , x_2 and x_3 represent different board features and w_0 , w_1 , w_2 and w_3 represent weights.

Choosing an Approximation Algorithm for the Target Function

The focus is to choose weights and fit the given training samples effectively. The aim is to reduce the error given as:

$$E \equiv \sum_{\substack{\text{Training} \\ \text{Samples}}} \left[V_{\text{train}}(b) - \hat{V}(b) \right]^2$$

Here, b is the sample and $\hat{V}(b)$ is the predicted hypothesis. The approximation is carried out as:

- Computing the error as the difference between trained and expected hypothesis. Let error be $error(b)$.
- Then, for every board feature x_i , the weights are updated as:

$$w_i = w_i + \mu \times error(b) \times x_i$$

Here, μ is the constant that moderates the size of the weight update.

Thus, the learning system has the following components:

- A Performance system to allow the game to play against itself.
- A Critic system to generate the samples.
- A Generalizer system to generate a hypothesis based on samples.
- An Experimenter system to generate a new system based on the currently learnt function. This is sent as input to the performance system.

3.4 INTRODUCTION TO CONCEPT LEARNING

Concept learning is a learning strategy of acquiring abstract knowledge or inferring a general concept or deriving a category from the given training samples. It is a process of abstraction and generalization from the data.

Concept learning helps to classify an object that has a set of common, relevant features. Thus, it helps a learner compare and contrast categories based on the similarity and association of positive and negative instances in the training data to classify an object. The learner tries to simplify by observing the common features from the training samples and then apply this simplified model to the future samples. This task is also known as learning from experience.

Each concept or category obtained by learning is a Boolean valued function which takes a true or false value. For example, humans can identify different kinds of animals based on common relevant features and categorize all animals based on specific sets of features. The special features that distinguish one animal from another can be called as a concept. This way of learning categories for object and to recognize new instances of those categories is called as concept learning. It is formally defined as inferring a Boolean valued function by processing training instances.

Concept learning requires three things:

1. Input – Training dataset which is a set of training instances, each labeled with the name of a concept or category to which it belongs. Use this past experience to train and build the model.
2. Output – Target concept or Target function f . It is a mapping function $f(x)$ from input x to output y . It is to determine the specific features or common features to identify an object. In other words, it is to find the hypothesis to determine the target concept. For e.g., the specific set of features to identify an elephant from all animals.
3. Test – New instances to test the learned model.

Formally, Concept learning is defined as—"Given a set of hypotheses, the learner searches through the hypothesis space to identify the best hypothesis that matches the target concept".

Consider the following set of training instances shown in Table 3.1.

Table 3.1: Sample Training Instances

S.No.	Horns	Tail	Tusks	Paws	Fur	Color	Hooves	Size	Elephant
1.	No	Short	Yes	No	No	Black	No	Big	Yes
2.	Yes	Short	No	No	No	Brown	Yes	Medium	No
3.	No	Short	Yes	No	No	Black	No	Medium	Yes
4.	No	Long	No	Yes	Yes	White	No	Medium	No
5.	No	Short	Yes	Yes	Yes	Black	No	Big	Yes

Here, in this set of training instances, the independent attributes considered are 'Horns', 'Tail', 'Tusks', 'Paws', 'Fur', 'Color', 'Hooves' and 'Size'. The dependent attribute is 'Elephant'. The target concept is to identify the animal to be an Elephant.

Let us now take this example and understand further the concept of hypothesis.

Target Concept: Predict the type of animal - For example –'Elephant'.

3.4.1 Representation of a Hypothesis

A *hypothesis* ' h ' approximates a target function ' f ' to represent the relationship between the independent attributes and the dependent attribute of the training instances. The hypothesis is the predicted approximate model that best maps the inputs to outputs. Each hypothesis is represented as a conjunction of attribute conditions in the antecedent part.

For example, $(\text{Tail} = \text{Short}) \wedge (\text{Color} = \text{Black}) \dots$

The set of hypothesis in the search space is called as hypotheses. Hypotheses are the plural form of hypothesis. Generally ' H ' is used to represent the hypotheses and ' h ' is used to represent a candidate hypothesis.

Each attribute condition is the constraint on the attribute which is represented as attribute-value pair. In the antecedent of an attribute condition of a hypothesis, each attribute can take value as either '?' or ' $\varnothing$ ' or can hold a single value.

- "?" denotes that the attribute can take any value [e.g., $\text{Color} = ?$]
- " $\varnothing$ " denotes that the attribute cannot take any value, i.e., it represents a null value [e.g., $\text{Horns} = \varnothing$]
- Single value denotes a specific single value from acceptable values of the attribute, i.e., the attribute 'Tail' can take a value as 'short' [e.g., $\text{Tail} = \text{Short}$]

For example, a hypothesis ' h ' will look like,

	Horns	Tail	Tusks	Paws	Fur	Color	Hooves	Size
$h =$	<No	?	Yes	?	?	Black	No	Medium>

Given a test instance x , we say $h(x) = 1$, if the test instance x satisfies this hypothesis h .

The training dataset given above has 5 training instances with 8 independent attributes and one dependent attribute. Here, the different hypotheses that can be predicted for the target concept are,

	Horns	Tail	Tusks	Paws	Fur	Color	Hooves	Size
$h =$	<No	?	Yes	?	?	Black	No	Medium>
(or)								
$h =$	<No	?	Yes	?	?	Black	No	Big>

The task is to predict the best hypothesis for the target concept (an elephant). The most general hypothesis can allow any value for each of the attribute.

It is represented as:

$\langle ?, ?, ?, ?, ?, ?, ?, ? \rangle$. This hypothesis indicates that any animal can be an elephant.

The most specific hypothesis will not allow any value for each of the attribute $\langle \phi, \phi, \phi, \phi, \phi, \phi, \phi, \phi \rangle$. This hypothesis indicates that no animal can be an elephant.

The target concept mentioned in this example is to identify the conjunction of specific features from the training instances to correctly identify an elephant.

Example 3.1: Explain Concept Learning Task of an Elephant from the dataset given in Table 3.1.

Given,

Input: 5 instances each with 8 attributes

Target concept/function 'c': Elephant $\rightarrow$ {Yes, No}

Hypotheses H : Set of hypothesis each with conjunctions of literals as propositions [i.e., each literal is represented as an attribute-value pair]

Solution: The hypothesis 'h' for the concept learning task of an Elephant is given as:

$h = \langle \text{No} \quad \text{Short} \quad \text{Yes} \quad ? \quad ? \quad \text{Black} \quad \text{No} \quad ? \rangle$

This hypothesis h is expressed in propositional logic form as below:

$(\text{Horns} = \text{No}) \wedge (\text{Tail} = \text{Short}) \wedge (\text{Tusks} = \text{Yes}) \wedge (\text{Paws} = ?) \wedge (\text{Fur} = ?) \wedge (\text{Color} = \text{Black}) \wedge (\text{Hooves} = \text{No}) \wedge (\text{Size} = ?)$

Output: Learn the hypothesis 'h' to predict an 'Elephant' such that for a given test instance x ,

$$h(x) = c(x)$$

This hypothesis produced is also called as concept description which is a model that can be used to classify subsequent instances.

Thus, concept learning can also be called as *Inductive Learning* that tries to induce a general function from specific training instances. This way of learning a hypothesis that can produce an approximate target function with a sufficiently large set of training instances can also approximately classify other unobserved instances and is called as inductive learning hypothesis. We can only determine an approximate target function because it is very difficult to find an exact target function with the observed training instances. That is why a hypothesis is an approximate target function that best maps the inputs to outputs.

3.4.2 Hypothesis Space

Hypothesis space is the set of all possible hypotheses that approximates the target function f . In other words, the set of all possible approximations of the target function can be defined as hypothesis space. From this set of hypotheses in the hypothesis space, a machine learning algorithm would determine the best possible hypothesis that would best describe the target function or best fit the outputs. Generally, a hypothesis representation language represents a larger hypothesis space. Every machine learning algorithm would represent the hypothesis space in a different manner about the function that maps the input variables to output variables. For example, a regression algorithm represents the hypothesis space as a linear function whereas a decision tree algorithm represents the hypothesis space as a tree.

The set of hypotheses that can be generated by a learning algorithm can be further reduced by specifying a language bias.

The subset of hypothesis space that is consistent with all-observed training instances is called as **Version Space**. Version space represents the only hypotheses that are used for the classification.

For example, each of the attribute given in the Table 3.1 has the following possible set of values.

Horns - Yes, No
 Tail - Long, Short
 Tusks - Yes, No
 Paws - Yes, No
 Fur - Yes, No
 Color - Brown, Black, White
 Hooves - Yes, No
 Size - Medium, Big

Considering these values for each of the attribute, there are $(2 \times 2 \times 2 \times 2 \times 2 \times 3 \times 2 \times 2) = 384$ distinct instances covering all the 5 instances in the training dataset.

So, we can generate $(4 \times 4 \times 4 \times 4 \times 4 \times 5 \times 4 \times 4) = 81,920$ distinct hypotheses when including two more values $[?, \emptyset]$ for each of the attribute. However, any hypothesis containing one or more $\emptyset$ symbols represents the empty set of instances; that is, it classifies every instance as negative instance. Therefore, there will be $(3 \times 3 \times 3 \times 3 \times 3 \times 4 \times 3 \times 3 + 1) = 8,749$ distinct hypotheses by including only '?' for each of the attribute and one hypothesis representing the empty set of instances. Thus, the hypothesis space is much larger and hence we need efficient learning algorithms to search for the best hypothesis from the set of hypotheses.

Hypothesis ordering is also important wherein the hypotheses are ordered from the most specific one to the most general one in order to restrict searching the hypothesis space exhaustively.

3.4.3 Heuristic Space Search

Heuristic search is a search strategy that finds an optimized hypothesis/solution to a problem by iteratively improving the hypothesis/solution based on a given heuristic function or a cost measure. Heuristic search methods will generate a possible hypothesis that can be a solution in

the hypothesis space or a path from the initial state. This hypothesis will be tested with the target function or the goal state to see if it is a real solution. If the tested hypothesis is a real solution, then it will be selected. This method generally increases the efficiency because it is guaranteed to find a better hypothesis but may not be the best hypothesis. It is useful for solving tough problems which could not be solved by any other method. The typical example problem solved by heuristic search is the travelling salesman problem.

Several commonly used heuristic search methods are hill climbing methods, constraint satisfaction problems, best-first search, simulated-annealing, A* algorithm, and genetic algorithms.

3.4.4 Generalization and Specialization

In order to understand about how we construct this concept hierarchy, let us apply this general principle of generalization/specialization relation. By generalization of the most specific hypothesis and by specialization of the most general hypothesis, the hypothesis space can be searched for an approximate hypothesis that matches all positive instances but does not match any negative instance.

Searching the Hypothesis Space

There are two ways of learning the hypothesis, consistent with all training instances from the large hypothesis space.

1. Specialization – General to Specific learning
2. Generalization – Specific to General learning

Scan for information on 'Additional Examples on Generalization and Specialization'



Generalization – Specific to General Learning This learning methodology will search through the hypothesis space for an approximate hypothesis by generalizing the most specific hypothesis.

Example 3.2: Consider the training instances shown in Table 3.1 and illustrate Specific to General Learning.

Solution: We will start from all false or the most specific hypothesis to determine the most restrictive specialization. Consider only the positive instances and generalize the most specific hypothesis. Ignore the negative instances.

This learning is illustrated as follows:

The most specific hypothesis is taken now, which will not classify any instance to true.

$h = \langle \varphi \quad \varphi \quad \varphi \quad \varphi \quad \varphi \quad \varphi \quad \varphi \quad \varphi \rangle$

Read the first instance I_1 , to generalize the hypothesis h so that this positive instance can be classified by the hypothesis h_1 .

I_1 : No Short Yes No No Black No Big Yes (Positive instance)

$h_1 = \langle \text{No} \quad \text{Short} \quad \text{Yes} \quad \text{No} \quad \text{No} \quad \text{Black} \quad \text{No} \quad \text{Big} \rangle$

When reading the second instance I_2 , it is a negative instance, so ignore it.

I_2 : Yes Short No No No Brown Yes Medium No (Negative instance)
 $h_2 = \langle \text{No Short Yes No No Black No Big} \rangle$

Similarly, when reading the third instance I_3 , it is a positive instance so generalize h_2 to h_3 to accommodate it. The resulting h_3 is generalized.

I_3 : No Short Yes No No Black No Medium Yes (Positive instance)
 $h_3 = \langle \text{No Short Yes No No Black No ?} \rangle$

Ignore I_4 since it is a negative instance.

I_4 : No Long No Yes Yes White No Medium No (Negative instance)
 $h_4 = \langle \text{No Short Yes No No Black No ?} \rangle$

When reading the fifth instance I_5 , h_4 is further generalized to h_5 .

I_5 : No Short Yes Yes Yes Black No Big Yes (Positive instance)
 $h_5 = \langle \text{No Short Yes ? ? Black No ?} \rangle$

Now, after observing all the positive instances, an approximate hypothesis h_5 is generated which can now classify any subsequent positive instance to true.

Specialization – General to Specific Learning This learning methodology will search through the hypothesis space for an approximate hypothesis by specializing the most general hypothesis.

Example 3.3: Illustrate learning by Specialization – General to Specific Learning for the data instances shown in Table 3.1.

Solution: Start from the most general hypothesis which will make true all positive and negative instances.

Initially,

$h = \langle ? ? ? ? ? ? ? \rangle$

h is more general to classify all instances to true.

I_1 : No Short Yes No No Black No Big Yes (Positive instance)

$h_1 = \langle ? ? ? ? ? ? ? \rangle$

I_2 : Yes Short No No No Brown Yes Medium No (Negative instance)

$h_2 = \langle \text{No ? ? ? ? ? ?} \rangle$

$\langle ? ? \text{Yes ? ? ? ?} \rangle$

$\langle ? ? ? ? ? \text{Black ?} \rangle$

$\langle ? ? ? ? ? ? \text{No} \rangle$

$\langle ? ? ? ? ? ? \text{Big} \rangle$

h_2 imposes constraints so that it will not classify a negative instance to true.

I_3 : No Short Yes No No Black No Medium Yes (Positive instance)

$h_3 = \langle \text{No ? ? ? ? ? ?} \rangle$

$\langle ? ? \text{Yes ? ? ? ?} \rangle$

	<?	?	?	?	?	Black	?	>
	<?	?	?	?	?	?	No	>
	<?	?	?	?	?	?	?	Big>
I4:	No	Long	No	Yes	Yes	White	No	Medium No (Negative instance)
h4 =	<?	?	Yes	?	?	?	?	>
	<?	?	?	?	?	Black	?	>
	<?	?	?	?	?	?	?	Big>

Remove any hypothesis inconsistent with this negative instance.

I5:	No	Short	Yes	Yes	Yes	Black	No	Big Yes (Positive instance)
h5 =	<?	?	Yes	?	?	?	?	>
	<?	?	?	?	?	Black	?	>
	<?	?	?	?	?	?	?	Big>

Thus, h_5 is the hypothesis space generated which will classify the positive instances to true and negative instances to false.

3.4.5 Hypothesis Space Search by Find-S Algorithm

Find-S algorithm is guaranteed to converge to the most specific hypothesis in H that is consistent with the positive instances in the training dataset. Obviously, it will also be consistent with the negative instances. Thus, this algorithm considers only the positive instances and eliminates negative instances while generating the hypothesis. It initially starts with the most specific hypothesis.

Algorithm 3.1: Find-S

Input: Positive instances in the Training dataset

Output: Hypothesis ' h '

1. Initialize ' h ' to the most specific hypothesis.

$h = \langle \varphi \quad \varphi \quad \varphi \quad \varphi \quad \varphi \quad \dots \rangle$

2. Generalize the initial hypothesis for the first positive instance [Since ' h ' is more specific].
3. For each subsequent instances:

If it is a positive instance,

Check for each attribute value in the instance with the hypothesis ' h '.

If the attribute value is the same as the hypothesis value, then do nothing,

Else if the attribute value is different than the hypothesis value, change it to '?' in ' h '.

Else if it is a negative instance,

Ignore it.

Example 3.4: Consider the training dataset of 4 instances shown in Table 3.2. It contains the details of the performance of students and their likelihood of getting a job offer or not in their final semester. Apply the Find-S algorithm.

Table 3.2: Training Dataset

CGPA	Interactiveness	Practical Knowledge	Communication Skills	Logical Thinking	Interest	Job Offer
≥ 9	Yes	Excellent	Good	Fast	Yes	Yes
≥ 9	Yes	Good	Good	Fast	Yes	Yes
≥ 8	No	Good	Good	Fast	No	No
≥ 9	Yes	Good	Good	Slow	No	Yes

Solution:

Step 1: Initialize ' h ' to the most specific hypothesis. There are 6 attributes, so for each attribute, we initially fill ' ϕ ' in the initial hypothesis ' h '.

$$h = \langle \phi \quad \phi \quad \phi \quad \phi \quad \phi \quad \phi \rangle$$

Step 2: Generalize the initial hypothesis for the first positive instance. I_1 is a positive instance, so generalize the most specific hypothesis ' h ' to include this positive instance. Hence,

$$I_1: \geq 9 \quad \text{Yes} \quad \text{Excellent} \quad \text{Good} \quad \text{Fast} \quad \text{Yes} \quad \text{Positive instance}$$

$$h = \langle \geq 9 \quad \text{Yes} \quad \text{Excellent} \quad \text{Good} \quad \text{Fast} \quad \text{Yes} \rangle$$

Step 3: Scan the next instance I_2 , since I_2 is a positive instance. Generalize ' h ' to include positive instance I_2 . For each of the non-matching attribute value in ' h ' put a '?' to include this positive instance. The third attribute value is mismatching in ' h ' with I_2 , so put a '?'.

$$I_2: \geq 9 \quad \text{Yes} \quad \text{Good} \quad \text{Good} \quad \text{Fast} \quad \text{Yes} \quad \text{Positive instance}$$

$$h = \langle \geq 9 \quad \text{Yes} \quad ? \quad \text{Good} \quad \text{Fast} \quad \text{Yes} \rangle$$

Now, scan I_3 . Since it is a negative instance, ignore it. Hence, the hypothesis remains the same without any change after scanning I_3 .

$$I_3: \geq 8 \quad \text{No} \quad \text{Good} \quad \text{Good} \quad \text{Fast} \quad \text{No} \quad \text{Negative instance}$$

$$h = \langle \geq 9 \quad \text{Yes} \quad ? \quad \text{Good} \quad \text{Fast} \quad \text{Yes} \rangle$$

Now scan I_4 . Since it is a positive instance, check for mismatch in the hypothesis ' h ' with I_4 . The 5th and 6th attribute value are mismatching, so add '?' to those attributes in ' h '.

$$I_4: \geq 9 \quad \text{Yes} \quad \text{Good} \quad \text{Good} \quad \text{Slow} \quad \text{No} \quad \text{Positive instance}$$

$$h = \langle \geq 9 \quad \text{Yes} \quad ? \quad \text{Good} \quad ? \quad ? \rangle$$

Now, the final hypothesis generated with Find-S algorithm is:

$$h = \langle \geq 9 \quad \text{Yes} \quad ? \quad \text{Good} \quad ? \quad ? \rangle$$

It includes all positive instances and obviously ignores any negative instance.

Limitations of Find-S Algorithm

1. Find-S algorithm tries to find a hypothesis that is consistent with positive instances, ignoring all negative instances. As long as the training dataset is consistent, the hypothesis found by this algorithm may be consistent.
2. The algorithm finds only one unique hypothesis, wherein there may be many other hypotheses that are consistent with the training dataset.

- Many times, the training dataset may contain some errors; hence such inconsistent data instances can mislead this algorithm in determining the consistent hypothesis since it ignores negative instances.

Hence, it is necessary to find the set of hypotheses that are consistent with the training data including the negative examples. To overcome the limitations of Find-S algorithm, Candidate Elimination algorithm was proposed to output the set of all hypotheses consistent with the training dataset.

3.4.6 Version Spaces

The version space contains the subset of hypotheses from the hypothesis space that is consistent with all training instances in the training dataset.

Scan for information on 'Additional Examples on Version Spaces'



List-Then-Eliminate Algorithm

The principle idea of this learning algorithm is to initialize the version space to contain all hypotheses and then eliminate any hypothesis that is found inconsistent with any training instances. Initially, the algorithm starts with a version space to contain all hypotheses scanning each training instance. The hypotheses that are inconsistent with the training instance are eliminated. Finally, the algorithm outputs the list of remaining hypotheses that are all consistent.

Algorithm 3.2: List-Then-Eliminate

Input: Version Space – a list of all hypotheses

Output: Set of consistent hypotheses

1. Initialize the version space with a list of hypotheses.
2. For each training instance,
 - remove from version space any hypothesis that is inconsistent.

This algorithm works fine if the hypothesis space is finite but practically it is difficult to deploy this algorithm. Hence, a variation of this idea is introduced in the Candidate Elimination algorithm.

Version Spaces and the Candidate Elimination Algorithm

Version space learning is to generate all consistent hypotheses around. This algorithm computes the version space by the combination of the two cases namely,

- Specific to General learning – Generalize S to include the positive example
- General to Specific learning – Specialize G to exclude the negative example

Using the Candidate Elimination algorithm, we can compute the version space containing all (and only those) hypotheses from H that are consistent with the given observed sequence of training instances. The algorithm defines two boundaries called '*general boundary*' which is a set of all hypotheses that are the most general and '*specific boundary*' which is a set of all hypotheses that are the most specific. Thus, the algorithm limits the version space to contain only those hypotheses that are most general and most specific. Thus, it provides a compact representation of List-then algorithm.

Algorithm 3.3: Candidate Elimination

Input: Set of instances in the Training dataset

Output: Hypothesis G and S

1. Initialize G , to the maximally general hypotheses.
2. Initialize S , to the maximally specific hypotheses.
 - Generalize the initial hypothesis for the first positive instance.
3. For each subsequent new training instance,
 - If the instance is **positive**,
 - o Generalize S to include the positive instance,
 - Check the attribute value of the positive instance and S ,
 - If the attribute value of positive instance and S are different, fill that field value with '?'.
 - If the attribute value of positive instance and S are same, then do no change.
 - o Prune G to exclude all inconsistent hypotheses in G with the positive instance.
 - If the instance is **negative**,
 - o Specialize G to exclude the negative instance,
 - Add to G all minimal specializations to exclude the negative example and be consistent with S .
 - If the attribute value of S and the negative instance are different, then fill that attribute value with S value.
 - If the attribute value of S and negative instance are same, no need to update ' G ' and fill that attribute value with '?'.
 - o Remove from S all inconsistent hypotheses with the negative instance.

Generating Positive Hypothesis ' S ' If it is a positive example, refine S to include the positive instance. We need to generalize S to include the positive instance. The hypothesis is the conjunction of ' S ' and positive instance. When generalizing, for the first positive instance, add to S all minimal generalizations such that S is filled with attribute values of the positive instance. For the subsequent positive instances scanned, check the attribute value of the positive instance and S obtained in the

previous iteration. If the attribute values of positive instance and S are different, fill that field value with a '?'. If the attribute values of positive instance and S are same, no change is required.

If it is a negative instance, it skips.

Generating Negative Hypothesis 'G' If it is a negative instance, refine G to exclude the negative instance. Then, prune G to exclude all inconsistent hypotheses in G with the positive instance. The idea is to add to G all minimal specializations to exclude the negative instance and be consistent with the positive instance. Negative hypothesis indicates general hypothesis.

If the attribute values of positive and negative instances are different, then fill that field with positive instance value so that the hypothesis does not classify that negative instance as true. If the attribute values of positive and negative instances are same, then no need to update 'G' and fill that attribute value with a '?'.

Generating Version Space – [Consistent Hypothesis] We need to take the combination of sets in 'G' and check that with 'S'. When the combined set fields are matched with fields in 'S', then only that is included in the version space as consistent hypothesis.

Example 3.4: Consider the same set of instances from the training dataset shown in Table 3.3 and generate version space as consistent hypothesis.

Solution:

Step 1: Initialize 'G' boundary to the maximally general hypotheses,

$$G = \langle ? \quad ? \quad ? \quad ? \quad ? \quad ? \rangle$$

Step 2: Initialize 'S' boundary to the maximally specific hypothesis. There are 6 attributes, so for each attribute, we initially fill ' ϕ ' in the hypothesis 'S'.

$$S = \langle \phi \quad \phi \quad \phi \quad \phi \quad \phi \quad \phi \rangle$$

Generalize the initial hypothesis for the first positive instance. I_1 is a positive instance; so generalize the most specific hypothesis 'S' to include this positive instance. Hence,

I_1 : ≥ 9 Yes Excellent Good Fast Yes **Positive instance**

$$S_1 = \langle \geq 9 \quad Yes \quad Excellent \quad Good \quad Fast \quad Yes \rangle$$

$$G_1 = \langle ? \quad ? \quad ? \quad ? \quad ? \quad ? \rangle$$

Step 3:

Iteration 1

Scan the next instance I_2 . Since I_2 is a positive instance, generalize ' S_1 ' to include positive instance I_2 . For each of the non-matching attribute value in ' S_1 ', put a '?' to include this positive instance. The third attribute value is mismatching in ' S_1 ' with I_2 , so put a '?'.

I_2 : ≥ 9 Yes Good Good Fast Yes **Positive instance**

$$S_2 = \langle \geq 9 \quad Yes \quad ? \quad Good \quad Fast \quad Yes \rangle$$

Prune G_1 to exclude all inconsistent hypotheses with the positive instance. Since G_1 is consistent with this positive instance, there is no change. The resulting G_2 is,

$$G_2 = \langle ? \quad ? \quad ? \quad ? \quad ? \quad ? \rangle$$

Iteration 2

Now Scan I_3 ,

I_3 : ≥ 8 No Good Good Fast No **Negative instance**

Since it is a negative instance, specialize G_2 to exclude the negative example but stay consistent with S_2 . Generate hypothesis for each of the non-matching attribute value in S_2 and fill with the attribute value of S_2 . In those generated hypotheses, for all matching attribute values, put a '?'. The first, second and 6th attribute values do not match, hence '3' hypotheses are generated in G_3 .

There is no inconsistent hypothesis in S_2 with the negative instance, hence S_3 remains the same.

$G_3 = < \geq 9$	?	?	?	?	?	>
<?	Yes	?	?	?	?	>
<?	?	?	?	?	?	Yes>
$S_3 = < \geq 9$	Yes	?	Good	Fast	Yes>	

Iteration 3

Now Scan I_4 . Since it is a positive instance, check for mismatch in the hypothesis ' S_3 ' with I_4 . The 5th and 6th attribute value are mismatching, so add '?' to those attributes in ' S_4 '.

I_4 : ≥ 9 Yes Good Good Slow No **Positive instance**

$S_4 = < \geq 9$ Yes ? Good ? ?>

Prune G_3 to exclude all inconsistent hypotheses with the positive instance I_4 .

$G_3 = < \geq 9$	?	?	?	?	?	>
<?	Yes	?	?	?	?	>
<?	?	?	?	?	?	Yes> Inconsistent

Since the third hypothesis in G_3 is inconsistent with this positive instance, remove the third one. The resulting G_4 is,

$G_4 = < \geq 9$	?	?	?	?	?	>
<?	Yes	?	?	?	?	>

Using the two boundary sets, S_4 and G_4 , the version space is converged to contain the set of consistent hypotheses.

The final version space is,

$< \geq 9$	Yes	?	?	?	?	>
$< \geq 9$	?	?	Good	?	?	>
<?	Yes	?	Good	?	?	>

Thus, the algorithm finds the version space to contain only those hypotheses that are most general and most specific.

The diagrammatic representation of deriving the version space is shown in Figure 3.2.

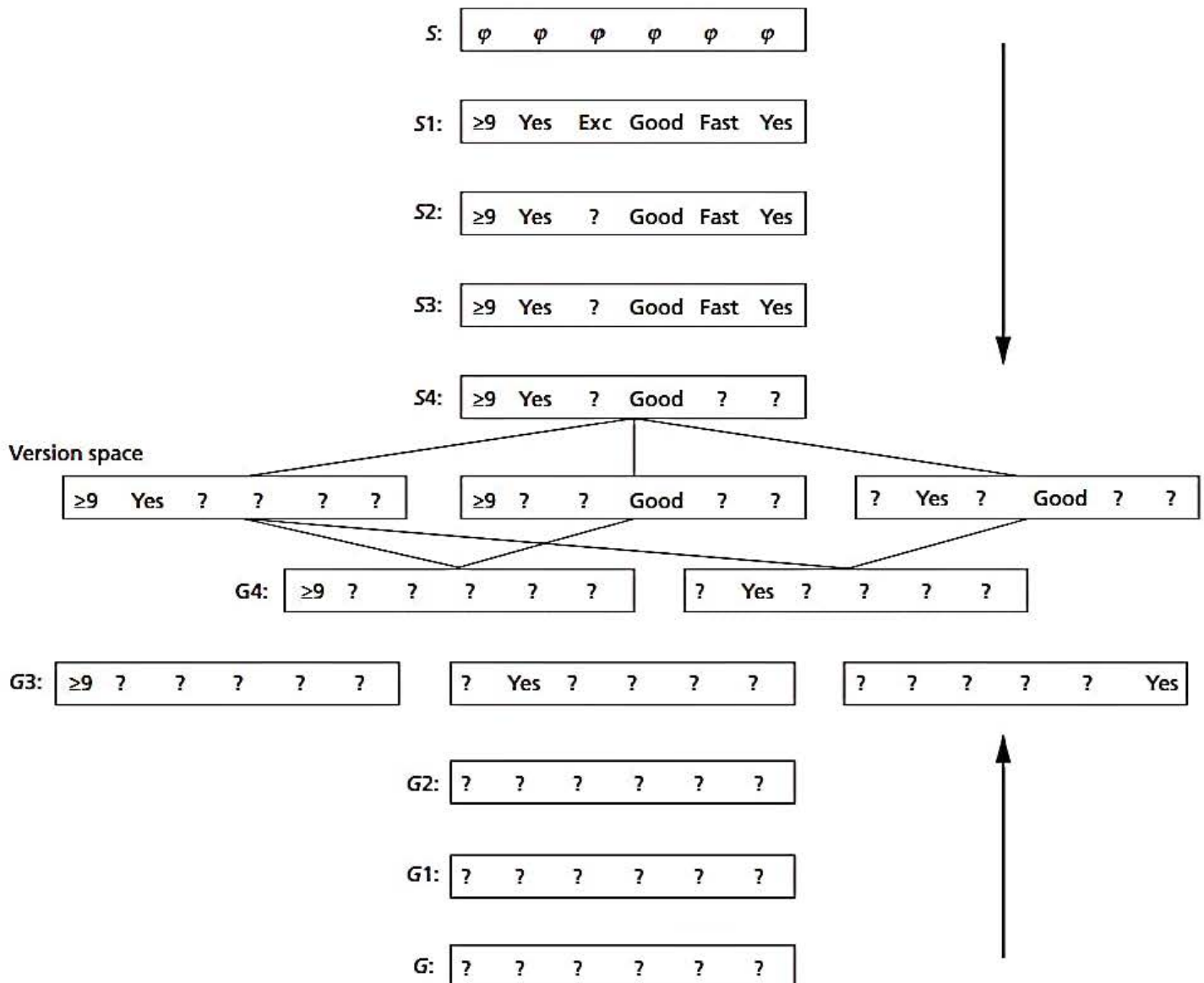


Figure 3.2: Deriving the Version Space

3.5 INDUCTION BIASES

Induction is a process of learning a target function or generalizing training data into a general model. **Inductive bias** is the set of prior assumptions considered by a learning algorithm beyond the training data, in order to perform induction. It is also called as the *bias* of the algorithm. It is similar to prior knowledge used for learning new concepts.

For example, linear regression learning assumes that the predictors (independent variables) are related to the target variable. Similarly, C4.5 decision tree learning algorithm assumes to always choose greedy best attributes as split criterion when constructing the decision tree. The k -nearest neighbour classifier assumes the neighbour instances in Euclidean distances belong to the same class and Naive Bayes classifier assumes that all input variables are independent variables, and so on.

There are two types of bias:

1. Constraint or Restriction – limit the hypothesis space
2. Preference – impose ordering on hypothesis space

So, when we select a hypothesis space, assumptions can be made by either imposing some restrictions or adding a preference to the algorithm. Concept learning is like refining the Hypothesis Space. It is a task of searching a hypothesis space of possible representations looking for the representation(s) that best fits the data, given the bias.

3.5.1 Bias and Variance

In supervised machine learning models, we need to find the target function $f(x)$ for an input value ' x ' that best maps the predicted output values to actual output values. If the predicted output value deviates from the actual output value, then we call it an error. There are three kinds of prediction errors—Bias error, Variance error and Irreducible error.

1. Irreducible errors cannot be reduced or avoided which normally happens because of various factors like unknown variables, noise, etc. These errors can sometimes be avoided by data cleaning.
2. A Bias error is the difference between the predicted output value and the actual output value of any learning model. Bias defines the accuracy of model predictions or the Mean Square Error (MSE) in the predictions. This bias error occurs due to inaccurate and simplifying assumptions or under-fitting during learning with the training data. The error is prominent when the test data is provided to the learned model.
3. A Variance error is the change in the target function sensitive to outliers and noise or small fluctuation in the training dataset. This change in the estimated target function occurs due to outliers, noise and when the number and types of parameters used to derive the mapping function change with different training sets. On the other hand, variance is the variations or spread from the actual value that can be seen between many models' predictions estimated from different training sets.

Pictorial representation of bias and variance is shown in Figure 3.3.

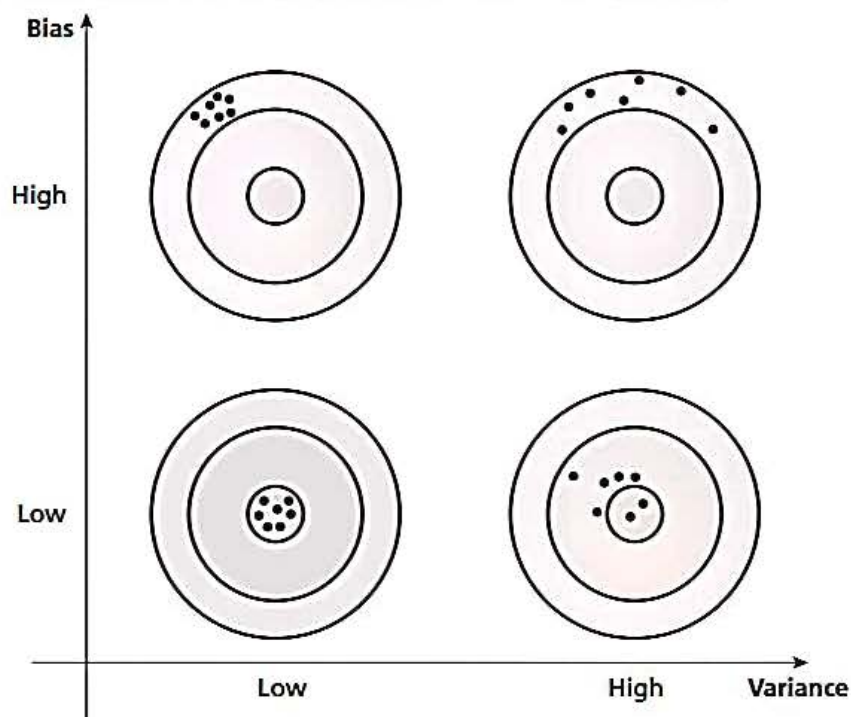


Figure 3.3: Bias vs Variance

3.5.2 Bias vs Variance Tradeoff

There always exists a tradeoff between bias and variance. Table 3.3 lists the differences and tradeoff between the two factors and how the performance of the different machine learning models differs in terms of bias and variance.

Table 3.3: Tradeoff between Bias and Variance

Bias	Variance
Low Bias algorithms have less simplifying assumptions. Machine learning models: Decision Trees, k-Nearest Neighbors (k-NN) and Support Vector Machines (SVM)	Low variance occurs when training data is general. But low variance algorithms have a slight difference in the estimated target function when training dataset changes. Machine learning models: Linear Regression, Linear Discriminant Analysis (LDA) and Logistic Regression
High Bias algorithms have more simplifying assumptions or considers the features that are not relevant. They underfit with the training data. Machine learning models: Parametric algorithms, Linear algorithms – Linear Regression, Linear Discriminant Analysis and Logistic Regression, Naïve Bayes algorithm	High variance occurs when training data is more sensitive to outliers and noise. High variance algorithms have more difference in the estimated target function when the training dataset changes. Machine learning models: Non-linear algorithms - Decision Trees, k-Nearest Neighbors and Support Vector Machines, Non-parametric algorithms
Linear algorithms are fast learning due to simplifying assumptions and hence exhibit high bias.	Decision tree algorithms have high variance, when the tree is grown very deep and they are not pruned before use.
High bias algorithms are consistent but inaccurate on average.	High variance algorithms are accurate on average but are inconsistent.

Linear machine learning algorithms often exhibit a high bias and a low variance, whereas **non-linear** machine learning algorithms exhibit a low bias but a high variance. We cannot minimize both bias and variance at the same time. Increasing the bias will decrease the variance and increasing the variance will decrease the bias.

Low Bias - High Variance algorithms are generally over-fitting and perform well with training dataset but are inconsistent for test data. On the other hand, the predictions that are made by a High Bias - Low Variance algorithms are inaccurate. Overfitting can be prevented by applying a process of regularization which regularizes or shrinks the coefficients towards zero by adding a penalty term called lambda λ to the error function. To deal with the high bias problem, we can add additional features such as adding polynomial features or decreasing regularization lambda λ , etc. To deal with the high variance problem we can add more training instances, reduce to smaller sets of features and increase regularization lambda λ .

One way of reducing the variance is by regularization which reduces the number of parameters during the training phase for parametric machine learning algorithms or by early stopping/pruning for tree-based algorithms and dropout for neural networks, etc.

Ensembling machine learning models is an approach to minimize bias and variance and to build accurate models that avoid over-fitting and under-fitting of learning models. Cross-validation can also be done to train on many datasets and average their predictions to ensemble different predictions from many models. Ensembling techniques are discussed in Chapter 12.

3.5.3 Best Fit in Machine Learning

Generalizing the hypothesis from training instances to a specific model is called inductive learning. Generalization basically describes the model's ability to infer or predict correctly a new unseen data after being trained with a training dataset. If a model is over trained or under trained, then its predictions are not going to be accurate.

Generally, performance of a machine learning model or predictions deteriorate when it learns too much or too less from the training instances. When a machine learning model learns too much from the training instances including noise, overfitting occurs. In other words, the model tries to fit the training data instances too well or predictors/features/independent variables are too complex. Moreover, when a model performs very well on the training data but poorly on the test data, then it is also an overfitting problem. Overfitting occurs more likely with non-parametric and non-linear models that have more flexibility when learning a target function. Underfitting generally occurs when a machine learning model could not learn from the training instances or the instances do not match with the model to learn or when predictors are very simple. In other words, the model does not fit the data well enough.

Ideally, the goal of selecting a machine learning model is to provide a performance in predictions between underfitting and overfitting. Hence, it is essential to find a model that provides a good fit but practically it is very difficult to achieve.

3.6 MODELLING IN MACHINE LEARNING

A machine learning model is an abstraction of the training dataset that can perform a prediction on new data. Training the model means feeding instances to the machine learning algorithm. Training datasets are used to fit and tune the model. After training a machine learning algorithm with the training data, a predictive model is generated as output to which a new data is fed to make predictions.

The process of modelling means training a machine learning algorithm with the training dataset, tuning it to increase performance, validating it and making predictions for a new unseen data. The major concern in machine learning is what model to select, how to train the model, time required to train, the dataset to be used, what performance to expect, and so on.

Learning the parameters is the main goal in machine learning algorithms. There are two types of parameters – model parameters and hyperparameters. Certain parameters can be learnt directly from training data and are called model parameters. For example, the coefficients used in

regression model, split attributes in decision tree model, weights and biases in neural networks and so on. Hyperparameters are higher-level parameters which cannot be learnt directly. For example, regularization lambda λ used in regularized regression, number of decision trees to include in a random forest, and so on.

Evaluating the selected machine learning model is also equally important as training the model. Hence, the dataset is split into two subsets called training dataset and test dataset, wherein the training dataset is used to train the model and the test dataset is used to evaluate the model. During training, the test dataset is unseen to the model so that the model can be tested properly on its ability to predict a new data. If the training and test datasets are the same, then the model can overfit but it would perform poorly when given a new unseen data.

During prediction, an error occurs when the estimated output does not match with the true output. Training error, also called as in-sample error, results when applying the predicted model on the training data, while Test error also called as out-of-sample error is the average error when predicting on unseen observations. The error function or the loss function is the aggregation of the differences between the true values and the predicted values. This loss function is defined as the **Mean Squared Error (MSE)**, which is the average of the squared differences between the true values Y_i and the predicted values $f(X_i)$ for an input value ' X_i '. A smaller value of MSE denotes that the error is less and, therefore, the prediction is more accurate.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N [Y_i - f(X_i)]^2$$

Machine Learning Process

The four basic steps in the machine learning process are:

1. Choose a machine learning algorithm to suit the training data and the problem domain
2. Input the training dataset and train the machine learning algorithm to learn from the data and capture the patterns in the data
3. Tune the parameters of the model to improve the accuracy of learning of the algorithm
4. Evaluate the learned model once the model is built

3.6.1 Model Selection and Model Evaluation

The biggest challenge in machine learning is choosing an algorithm that suits the problem. Hence, model selection and assessment are very important and deal with two types of complexities.

1. Model Performance – How well the model performs on the training dataset?
2. Model Complexity – How much complexity the model possesses after the training phase is over?

Model Selection is a process of selecting one good enough model among different machine learning models for the dataset or selecting different sets of features or hyperparameters for the same machine learning model. It is difficult to find the best model because all models exhibit some predictive error for the problem, so atleast a good enough model should be selected that performs fairly well with the dataset.

Some of the approaches used for selecting a machine learning model are listed below:

1. Use resample methods and split the dataset as training, testing and validation datasets and observe the performance of the model over all the phases. This approach is suitable for smaller datasets.
2. The simplest approach is to fit a model on the training dataset and to compute measures like error or accuracy.
3. The use of probabilistic framework and quantification of the performance of the model as a score is the third approach.

These methods are discussed in the following sections.

3.6.2 Re-sampling Methods

Re-sampling is a technique to select a model by reconstructing the training dataset and test dataset by randomly choosing instances by some method from the given dataset. This method involves selecting different instances repeatedly from a training dataset to tune a model. It is done to improve the accuracy of a model. The common re-sampling model selection methods are Random train/test splits, Cross-Validation (K -fold, LOOCV, etc.) and Bootstrap.

Cross-Validation

Cross-Validation is a method by which we can tune the model with only training dataset. It is a model evaluation approach by which we can set aside some data of the training dataset for validation and fit the rest of the data to train the model. The best model is found by estimating the average of errors on different test data. The popular cross-validation family of methods includes Holdout method, K -fold cross-validation, Stratified cross-validation and Leave-One-Out Cross-Validation (LOOCV).

Holdout Method

This is the simplest method of cross-validation. The dataset is split into two subsets called training dataset and test dataset. The model is trained using the training dataset and then evaluated using the test dataset. This holdout method can be applied for a single time which is called as single holdout method or it can be repeated for more than once which is called as repeated holdout method. The average performance on the test dataset is estimated to evaluate the model. Even though this model is very simple, it can exhibit high variance and the performance largely depends on how the dataset is split.

K-fold Cross-Validation

Another way of cross-validating is using a k -fold cross-validation, which will split the training dataset into k equal folds/parts creating $k - 1$ subsets of training set and one test subset. Out of the k folds, $k - 1$ folds are used for training and one fold is used for testing the model. This has to be performed for k iterations and during each iteration a different fold is selected for testing. The average performance of the model on k iterations is the final estimate of the model performance.

The illustration of this re-sampling is shown in Figure 3.4.

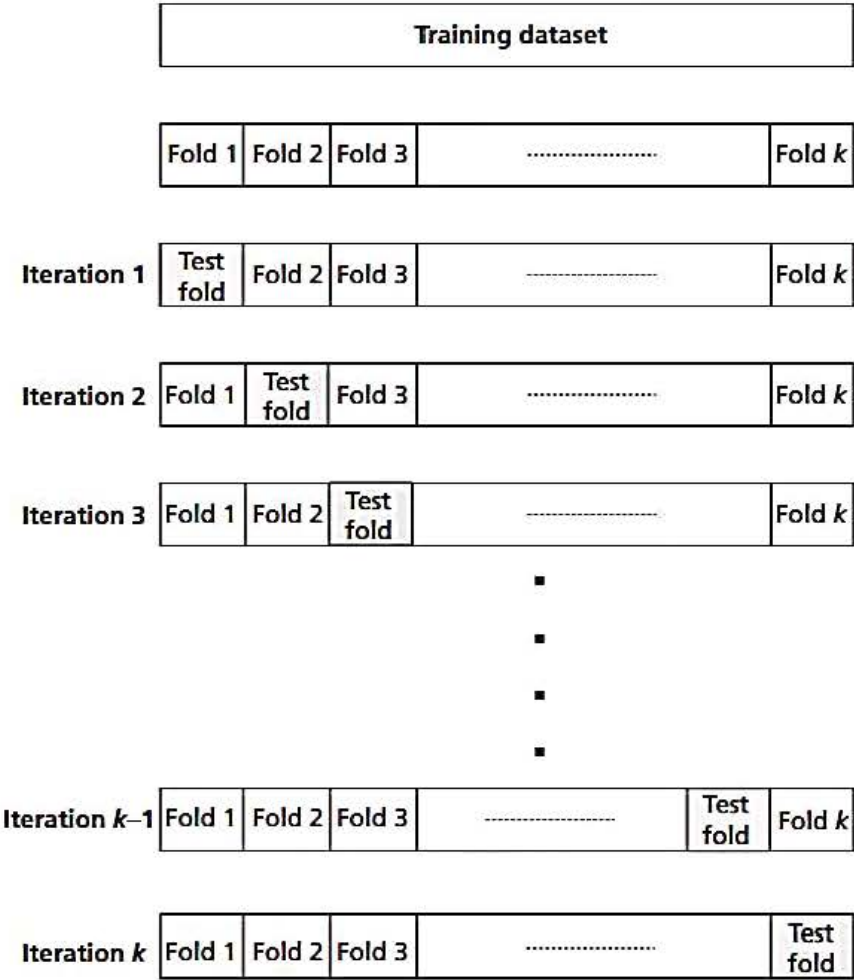


Figure 3.4: Illustration of K-fold Cross-Validation

Algorithm 3.4: K-fold Cross Validation

- Input:** Dataset with ' n ' instances
' k ' – Number of folds
- Repeat the following steps for ' k ' times with a different 'hold-out' fold for evaluation:
 - Split the dataset into k equal folds.
 - Train the model on $(k - 1)$ folds.
 - Evaluate it on the 1 remaining "hold-out" fold.
 - Compute average of the performance across all k hold-out folds.

Stratified K-fold Cross-Validation

This method is similar to k -fold cross-validation but with a slight difference. Here, it is ensured that while splitting the dataset into k folds, each fold should contain the same proportion of instances with a given categorical value. This is called stratified cross-validation.

Leave-One-Out Cross-Validation (LOOCV)

This method repeatedly splits the n data instances of the dataset into training dataset containing $n - 1$ data instances and leaving one data instance for evaluating the model. This process is repeated n times and average test error is then estimated for the model. Even though this model is expensive and time consuming because it has to run for n times (i.e., n data instances in the dataset), it has less bias. For example, if the training dataset contains 100 data instances, then 99 instances are used for training and one instance to test or evaluate the model. This process is repeated 100 times selecting a different instance as holdout instance for testing in each iteration.

The illustration of this re-sampling is shown in Figure 3.5.

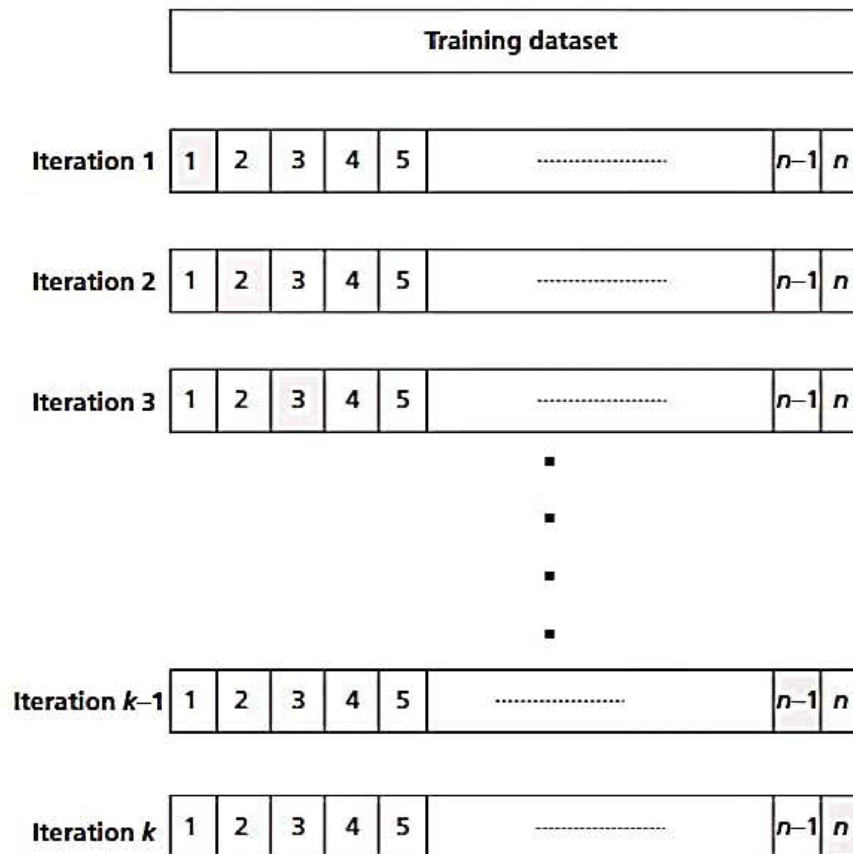


Figure 3.5: Illustration of Leave-One-Out Cross-Validation

Algorithm 3.5: LOOCV

Input Dataset with ' n ' instances:

1. Repeat the following steps for ' n ' times with a different 'hold-out' data instance for evaluation:
 - Train the model on $(n - 1)$ data instances.
 - Evaluate it on the one remaining 'hold-out' test instance.
2. Compute average of the performance across all n holdouts.

Model Performance

Classifier models are discussed in the subsequent chapters. The focus of this section is the evaluation of classifier models. Classifiers are unstable as a small change in the input can change the output. A solid framework is needed for proper evaluation. There are several metrics that can be used to describe the quality and usefulness of a classifier. One way to compute the metrics is to form a table called contingency table. For example, consider a test for detecting a disease, say cancer. Table 3.4 shows a contingency table for this scenario.

Table 3.4: Contingency Table

Test vs Disease	Has Disease Cancer	Has No Disease As Cancer
Positive	True Positive	False Positive
Negative	False Negative	True Negative

In this table, True Positive (TP) = Number of cancer patients who are classified by the test correctly, True Negative (TN) = Number of normal patients who do not have cancer are correctly detected. The two errors that are involved in this process is False Positive (FP) that is an alarm that indicates that the tests show positive when the patient has no disease and False Negative (FN) is another error that says a patient has cancer when tests says negative or normal. FP and FN are costly errors in this classification process.

The metrics that can be derived from this contingency table are listed below:

1. Sensitivity – The sensitivity of a test is the probability that it will produce a true positive result when used on a test dataset. It is also known as true positive rate. The sensitivity of a test can be determined by calculating:

$$\frac{TP}{TP + FN}$$

2. Specificity – The specificity of a test is the probability that a test will produce a true negative result when used on test dataset.

$$\frac{TN}{TN + FP}$$

3. Positive Predictive Value – The positive predictive value of a test is the probability that an object is classified correctly when a positive test result is observed.

$$\frac{TP}{TP + FP}$$

4. Negative Predictive Value – The negative predictive value of a test is the probability that an object is not classified properly when a negative test result is observed.

$$\frac{TN}{TN + FN}$$

5. Accuracy – The accuracy of the classifier can be shown in terms of sensitivity computed as:

$$\frac{TP + TN}{TP + TN + FP + FN}$$

6. Precision – Precision is also known as positive predictive power. It is defined as the ratio of true positive divided by the sum of true positive and false positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision indicates how good classifier is in predicting the positive classes.

7. Recall – It is same as sensitivity.

$$\text{Recall} = \text{Sensitivity} = \frac{TP}{TP + FN}$$

A combination of harmonic mean of precision and recall is called *F-measure* or *F1 score*. This is useful in identifying the model skill for a specific threshold.

Classifier Performance as Distance Measures The classifier performance can be computed as a distance measure also. The classifier accuracy can be plotted as a point. A point in the north-west is a better classifier. Euclid distance of two points of the two classifiers can give a performance measure. The value ranges from 0 to 1.

Visual Classifier Performance Receiver Operating Characteristic (ROC) curve and Precision-Recall curves indicate the performance of classifiers visually. ROC curves are visual means of checking the accuracy and comparison of classifiers. ROC is a plot of sensitivity (True Positive Rate) and the 1-specificity (False Positive Rate) for a given model.

A sample ROC curve is shown in Figure 3.6, where results of five classifiers are given. A is the ROC of an average classifier. The ideal classifier is E where the area under curve is 1.0. Theoretically, it can range from 0.9 to 1. The rest of the classifiers B, C, D are categorized based on area under curve as good, better and still better based on the area under curve values.

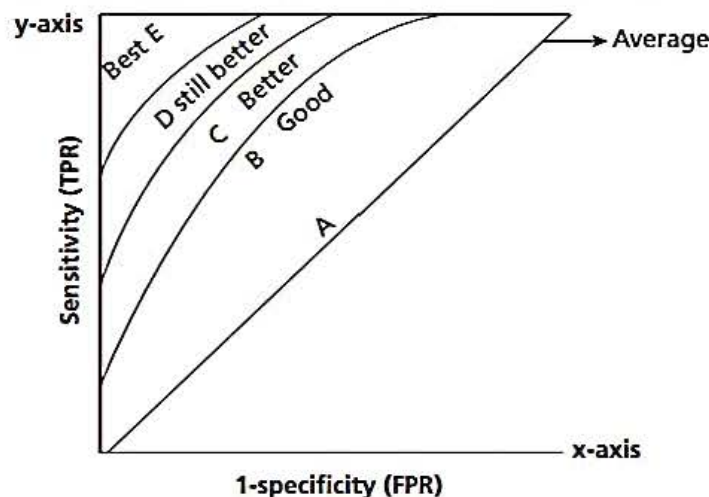


Figure 3.6: A Sample ROC Curve

The classifier prediction is based on the threshold value. A threshold value of 0.5 maps the probability to class 0 or class 1. The threshold value can be tuned to control the higher or lower FP and FN. This is useful when one focuses on error – FP or FN for an experiment.

We start from the bottom left-hand corner initially. If we have any true positive case, we move up and plot a point. If it is a false positive case, we move right and plot. This process is repeated until the complete curve is drawn. In ROC, the diagonal of the plot indicates the model has no skill or random classifier, and skillful models show the curve above the diagonal. In short, if the ROC curve is closer to the diagonal line, then it shows the classifier to be less accurate.

Instead of predicting the label of a classifier, one can predict the probabilities of the model. Probabilities allow some better evaluation by functions that are called scoring functions or scoring rules. The area under curve (AUC) is one such score that can be used for classifier model evaluation. The integrated AUC is a measure of the model across threshold values.

AUC indicates the accuracy of the model. A model is perfect if it has area under ROC curve as one. The AUC score 0 of a model indicates the wrong model. The approximate area under precision-recall curve also indicates the power of the model across thresholds.

A precision-recall curve is a plot of precision and recall for different threshold values. This curve is useful if there is an imbalance in the classes where one class has more labels and other classes have less samples.

ROC is used when there is no class imbalance and precision-recall curves are used when there is a moderate-to-large class imbalance.

Scoring Methods

Another alternative for model selection is to combine the complexity of the model and performance of the model as a score. Then, model selection is done by selecting the model that maximizes or minimizes the score.

Minimum Description Length (MDL) is one such method. The aim is to describe target variable and model in terms of bits. MDL is the principle of using minimum number of bits to represent the data and model. It is a variant of Occam Razor's principle that states that the model with the simplest explanation is the best model. MDL too recommends the selection of the hypothesis that minimizes the sum of two descriptions of data and model.

Let h be a learning model. Let $L(h)$ is the number of bits used to represent the model and D is the number of predictions, then the MDL is given as:

$$L(h) + L(D|h)$$

where, $L(D|h)$ is the number of bits used to represent the predictions D based on the training set.

MDL can be expressed in terms of negative log-likelihood also as:

$$\text{MDL} = -\log(p(\Theta)) - \log(p(y | x, \Theta))$$

where, y is the target variable, x is the input and Θ is the model parameters.

3.7 LEARNING FRAMEWORKS

Some of the theoretical aspects of learning frameworks are discussed in the following sections.

3.7.1 PAC Framework

PAC learning is a framework that performs a mathematical analysis of machine learning models in choosing the probably approximate correct hypothesis with low error. One of the important learning framework is called Probably Approximate Correct (PAC) framework. This framework was developed by Leslie Valiant as theoretical machine framework and deals with the quantification of difficulty in learning tasks.

A hypothesis that returns wrong predictions most of the times can be rejected immediately. In the same manner, if the hypothesis returns correct answers consistently with large number of samples, it is unlikely to be wrong. In short, it must be probably approximately correct. Here in PAC learning, the word 'approximate' refers to the target function and 'probably' refers to the low generalization error. Any algorithm that returns a hypothesis that is PAC is called PAC-algorithm. The basic assumptions of PAC framework are given below:

1. The unseen samples are drawn from the past samples using a fixed distribution.
2. The rate of samples that are classified wrongly is called an error rate. Normally, Boolean functions with the loss rate on 0 – 1 can be considered as:

$$error(h) = Loss_{0/1}(h) = \sum_{x,y} L_{0/1}(y, h(x))p(x, y)$$

Let hypothesis $f(x)$ exists although it is unknown for the given problem. The focus of PAC is to find a hypothesis that is a close match to the unknown hypothesis $f(x)$. A bad hypothesis can be found out easily based on the predictions of the new data, but a consistent hypothesis that returns correct answers over many samples must be true. This is the central premise of PAC framework. So, how many samples are required for the good generalization to determine whether the given hypothesis is PAC learnable or not?

The hypothesis can be called approximately correct if error is less than or equal to, ϵ . That is,

$$error(h) \leq \epsilon$$

Here, ϵ is small constant. The hypothesis is closer to the real hypothesis within ϵ -class in the hypothesis space. Let N be the samples and let h_b be the wrong hypothesis. Then, the probability that it agrees with the given example is $1 - \epsilon$. Therefore, the bound for N samples is:

$$P(h_b) \leq (1 - \epsilon)^N \text{ in agreement with } N \text{ samples}$$

The probability that it has at least one consistent hypothesis is bound by the sum of individual probabilities given as:

$$|H_{bad}|(1 - \epsilon)^N \leq |H|(1 - \epsilon)^N$$

The focus is to reduce the error, therefore, $|H|(1 - \epsilon)^N \leq \delta$. Here, δ is a small number. Therefore, the number of samples N can be determined as:

$$N \geq \frac{1}{\epsilon} \left(\ln \frac{1}{\delta} + \ln |H| \right)$$

In short, if the algorithm returns a consistent hypothesis with N samples, then the probability is at least $(1 - \delta)$ with an error at most ϵ . This is called sample complexity.

Mistake Bound Model

This is an alternate model of PAC where the learner is evaluated by the number of mistakes the algorithm makes before it converges to the correct hypothesis. The difference between this model and PAC is that, in mistake bound model, the learner predicts the target function $f(x)$ before it is shown the correct function. The number of mistakes is counted, which is important for applications that are critical. One can count the mistakes before being applied for PAC. For Find-S, the problem is given a concept c with a hypothesis of ' n ' variables. Initially $h(x)$ is given as $x_1 \bar{x}_1, x_2 \bar{x}_2, \dots, x_n \bar{x}_n$ as a list L . If the prediction made using $h(x)$ is false, but the label is true, then all literals in h that are

false are removed. If the prediction made using $h(x)$ is true but the label is wrong, then the output is inconsistent conjunction. This is repeated for all cases of predictions. It has been found that at most $n + 1$ errors can happen and hence the bound is given as $O(nL)$.

Similarly, for Find S and Halving algorithm, mistake bound model can make at most $\lceil \log_2 |H| \rceil$ mistakes before exact learning. Then, for an arbitrary class C , the worst cast algorithm makes opt C mistakes, as:

$$VC(C) \leq OPT(C) \leq \log_2 (|C|)$$

where, $VC(C)$ is the size of the largest subset that can be shattered by the hypothesis.

Finite and Infinite Hypothesis

The *Sample Complexity* of a machine learning algorithm defines the number of training instances required to converge to a successful hypothesis for a target function. On the other hand, *Computational Complexity* defines how much computational effort is required for the machine learning model to converge to a successful hypothesis for a target function.

In a finite hypothesis space, PAC framework can identify the classes of hypotheses that can be learned from a polynomial number of training samples, whereas in an infinite hypothesis space, it cannot be learned from a polynomial number of training samples. Therefore, for an infinite hypothesis space, VC dimension defines the boundary for the number of training instances required for inductive learning.

3.7.2 Estimating Hypothesis Accuracy

Evaluating the accuracy of the hypothesis ' h ' is very important in machine learning. There are some statistical methods to estimate the accuracy of the hypothesis. Sometimes, the hypothesis for a target function may outperform the training instances but when a test instance from the domain is given it would misclassify the instance. The *Sample Error* of the hypothesis is defined as the proportion of instances misclassified with the training instances. The *True Error* of the hypothesis is defined as the probability that the hypothesis will misclassify an instance drawn at random from the domain.

To estimate the True Error, confidence interval can be computed with the Sample Error, which would approximate it to 95%.

Given, n as the number of training instances and $error_s(h)$ as the Sample Error, confidence interval can be calculated as:

$$error_s(h) \pm 1.96 \sqrt{\frac{error_s(h) - (1 - error_s(h))}{n}}$$

3.7.3 Hoeffding's Inequality

Any learning algorithm initially considers a finite hypothesis set H from which it finds a hypothesis $h \in H$ to approximate an unknown target function f . Given a training data, this learned hypothesis h can perform well but with the whole dataset or with an unknown test data we need to measure the bias (error) between the target function f and the derived hypothesis h .

The error between the target function f and the hypothesis h with the training data or during the phase of modelling is called the In-Sample error/Empirical error. Similarly, the error between the target function f and the hypothesis h when an unknown test data is given

is called as Out-of-sample error/Generalization error. Obviously, this in-sample error is smaller than the out-of-sample error.

The relationship between the in-sample error and out-of-sample error can be quantified using probabilistic bound expressed with a powerful tool called Hoeffding's inequality.

Let us consider,

$h \in H$ as the derived hypothesis

f as the target function

E_{in} the in-sample error

E_{out} the out-of-sample error

n the size of the sample dataset

Hoeffding's inequality gives an upper bound for ' h ' in a finite hypothesis set as:

$$P(|E_{\text{in}}(h) - E_{\text{out}}(h)| \geq \varepsilon) \leq \delta = e^{-2n\varepsilon^2}$$

where, $\varepsilon > 0$ is some small value used to measure the deviation between E_{in} and E_{out} . However, this deviation is less than or equal to some bound δ which reduces exponentially as ε and/or the sample size n increases.

3.7.4 Vapnik–Chervonenkis Dimension

With an infinite hypothesis set, a generalization bound can be derived using Vapnik–Chervonenkis (VC) dimension. In order to determine whether the learned function ' h ' is PAC learnable and to derive the right bound, this VC dimension is required.

VC dimension of a classifier or a hypothesis ' h ' is defined to be the maximum number of data points ' n ' that can be shattered (classified/separated) into positive and negative data points for all possible combinations of the data points (i.e., to classify all possible 2^n labelling of data instances correctly).

For example, if there are 2 data points, a linear classifier would be able to shatter or classify correctly all 2^2 combinations of the data instances as shown in Figure 3.7. Similarly, for any 3 data points, a line would correctly shatter or classify all combinations of the 3 data instances (i.e., 2^3) that belongs to 2 classes (C_1, C_2) as shown in Figure 3.8.

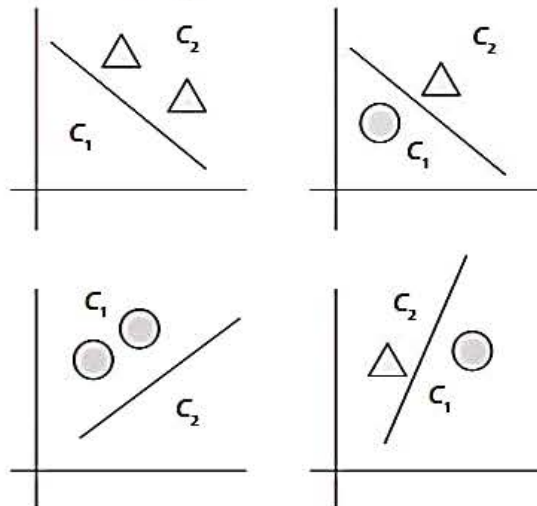


Figure 3.7: Shattering 2 Data Points

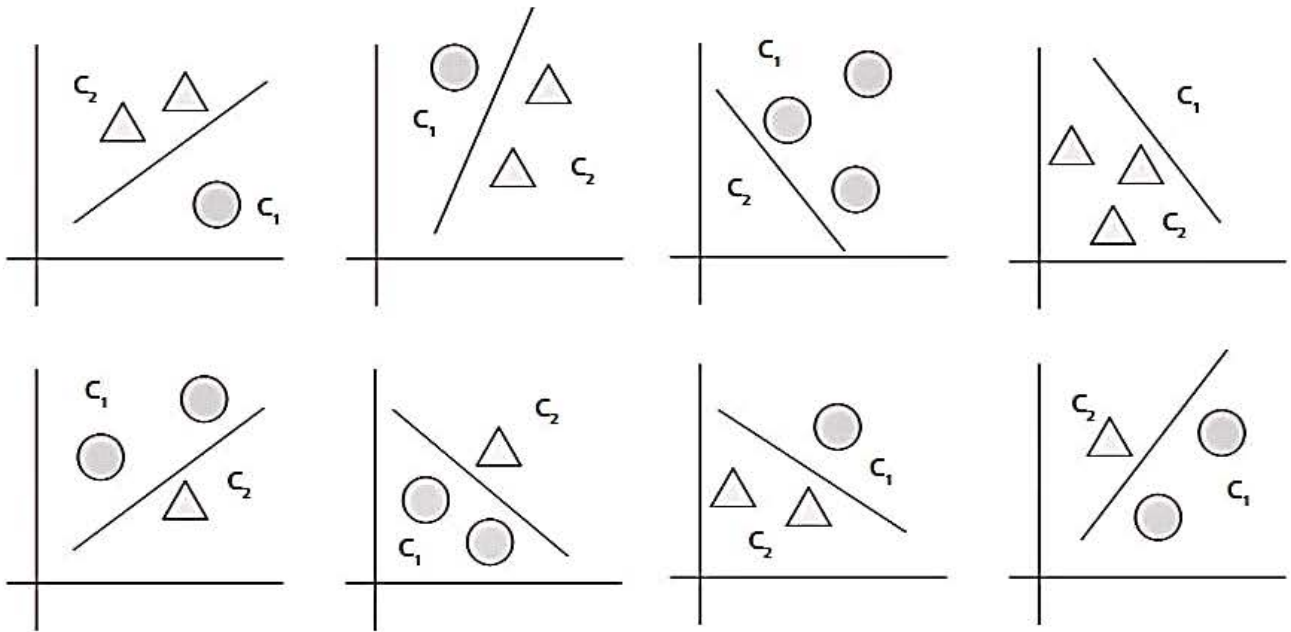


Figure 3.8: Shattering 3 Data Points

Certain representations like plane, hyper plane, square, rectangle, circle, etc. can also shatter larger sets of datapoints (which are more than 3) and they have higher VC dimension. Complex classifiers or learners such as perceptron, SVM, etc. can also classify more number of data points. On the other hand, for an infinite hypothesis space, VC dimension is almost like $\log_2(|h|)$.

With VC dimension, the generalization error of a hypothesis 'h' is bounded by:

$$E_{\text{out}}(h) \leq E_{\text{in}}(h) + \sqrt{\frac{8d \left(\ln \left(\frac{2m}{d} \right) + 1 \right) + 8 \ln \frac{4}{\delta}}{m}}$$

where, d denotes the VC dimension and m is the sample from the training set.

Thus, with the more complex model, we have higher VC dimension but the out-of-sample-error will be approximated by the in-sample error.

Summary

1. *Learning* is a process by which one can acquire knowledge and construct new ideas or concepts based on the experiences.
2. Concept learning is a learning strategy of acquiring abstract knowledge or inferring a general concept or deriving a category from the given training samples.
3. Each concept or category obtained by learning is a Boolean valued function which takes a true or false value.
4. A *hypothesis* 'h' is an approximation of a target function 'f' to represent the relationship between the independent attributes and the dependent attribute of the training instances.
5. The hypothesis is the predicted approximate model that best maps the inputs to outputs.
6. The most general hypothesis can allow any value for each of the attribute. It is represented as: $\langle ?, ?, ?, ?, ?, ?, ?, ? \rangle$.
7. The most specific hypothesis will not allow any value for any of the attribute. It is represented as, $\langle \phi, \phi, \phi, \phi, \phi, \phi, \phi, \phi \rangle$.

8. The hypothesis produced is also called as concept description which is a model that can be used to classify subsequent instances.
9. The set of all possible approximations of the target function is called as *Hypothesis space*.
10. The subset of hypothesis space that is consistent with all observed training instances is called as *Version Space*.
11. Hypothesis ordering is also important wherein the hypotheses are ordered from the most specific one to the most general, in order to restrict searching the hypothesis space exhaustively.
12. There are two ways of learning the hypothesis consistent with all training instances from the large hypothesis space called (a) general to specific, (b) specific to general.
13. Find-S algorithm is guaranteed to converge to the most specific hypothesis in H that is consistent with the positive instances in the training dataset.
14. List-Then-Eliminate Algorithm initializes the version space to contain all hypotheses and then eliminate any hypothesis that is found inconsistent with any training instance.
15. Candidate Elimination algorithm limits the version space to contain only those hypotheses that are most general and most specific.
16. Inductive Learning tries to induce a general function from specific training instances.
17. This way of learning a hypothesis that can produce an approximate target function with a sufficiently large set of training instances and can also approximately classify other unobserved instances is called as Inductive learning hypothesis.
18. There always exists a tradeoff between bias and variance.
19. When a machine learning model learns too much from the training instances including noise, overfitting occurs.
20. Underfitting generally occurs when a machine learning model could not learn from the training instances or when predictors are very simple.
21. The process of modelling means training a machine learning algorithm with the training dataset, tuning it to increase performance, validating it and making predictions for a new unseen data.
22. Evaluating the selected machine learning model is also equally important as training the model.
23. Model selection is a process of selecting one good enough model among different machine learning models for the dataset or selecting different sets of features or hyperparameters for the same machine learning model.
24. Re-sampling is a technique to do model selection by reconstructing the training dataset and test dataset by randomly selecting instances by some method from the given dataset.
25. Cross-Validation is a method by which we can tune the model with only training dataset.
26. The popular cross-validation family of methods includes Holdout method, K -fold cross-validation, Stratified cross-validation and Leave-One-Out Cross-Validation (LOOCV).
27. When learning hypothesis, there is a need to measure the bias (error) between the target function f and the derived hypothesis h .
28. The relationship between the in-sample error and out-of-sample error can be quantified by using probabilistic bound expressed with a powerful tool called Hoeffding's inequality.
29. In an infinite hypothesis set, a generalization bound can be derived using Vapnik-Chervonenkis (VC) dimension.

Key Terms

- **Computational Learning Theory (COLT)** – A field of study of theoretical aspects of machine learning.
- **Dataset** – A collection of data in tabular form with each column called as an attribute/variable/feature and each row called as a training instance/training sample.
- **Training Dataset** – A subset of instances from the dataset which is used for training the model.
- **Training Instance/Training Sample** – A single row of data in the training dataset.
- **Variable** – A single column of data in the training dataset.
- **Predictor Variables** – The variables which are used to predict the 'Target Variable'. These predictor variables are also called as independent variables.
- **Target Variable** – The output variable that needs to be predicted. The target variable is also called as dependent variable.
- **Target Function** – The mapping function between the independent variables and the dependent variable.
- **Hypothesis** – It is represented as ' h ' and is a function that approximates the target function.
- **Hypothesis Space** – It is represented as ' H ' is a set of hypotheses functions that are generated by a learner for approximating the target function.
- **Testing Dataset** – The subset of the dataset used to validate the accuracy of the machine learning model but not used to train the model. Sometimes, it is also called as validation dataset.
- **Inductive Learning** – Induces a general function from specific training instances.
- **Version Space** – It is the subset of hypotheses from the hypothesis space that are consistent with all training instances in the training dataset.
- **Induction** – A process of learning a target function or generalizing training data into a general model.
- **Inductive Bias** – The set of prior assumptions considered by a learning algorithm beyond the training data in order to perform induction. It is also called as the bias of the algorithm.
- **Irreducible Errors** – Cannot be reduced or avoided which normally happens because of various factors like unknown variables, noise, etc.
- **Bias Error** – The difference between the predicted output value and the actual output value of any learning model.
- **Variance Error** – The variations or spread from the actual value that can be seen between many models' predictions estimated from different training sets.
- **Training Error** – Also called as in-sample error is the error that results when applying the predicted model on the training data.
- **Test Error** – Also called as out-of-sample error is the average error when predicting on unseen observations.
- **In-Sample Error/ Empirical Error** – The error between the target function f and the hypothesis h with the training data or during the phase of modelling.
- **Out-of-Sample Error/Generalization Error** – The error between the target function f and the hypothesis h with an unknown test data.
- **Sensitivity** – Sensitivity measures the proportion of positives that are correctly identified in a binary classification test.

- **Specificity** – Specificity measures the proportion of negatives that are correctly identified in a binary classification.
- **ROC** – Receiver Operated Characteristic curve is a plot of sensitivity and FP-rate for a given model.
- **PAC (Probably Approximate Correct) Learning** – A framework that performs a mathematical analysis of machine learning models in choosing the probably approximate correct hypothesis with low error.
- **Sample Complexity** – Defined as the number of training instances required to converge to a successful hypothesis for a target function.
- **Computational Complexity** – Defined as how much computational effort is required for the machine learning model to converge to a successful hypothesis for a target function.
- **Sample Error of the Hypothesis** – Defined as the proportion of instances misclassified with the training instances.
- **True Error of the Hypothesis** – Defined as the probability that the hypothesis will misclassify an instance drawn at random from the domain.

Review Questions

1. What are the different methods by which learning happens?
2. Define hypothesis and hypothesis space.
3. What are predictor variables and target variables?
4. What is a target function?
5. Define Concept learning.
6. What do you understand by a Concept?
7. What are the three things required for Concept learning?
8. How is a hypothesis represented?
9. Consider the following training dataset given in Table 3.5 which consists of 6 instances.

Table 3.5: Training Dataset

S.No.	Horns	Tail	Tusks	Paws	Fur	Color	Hooves	Size	Elephant
1.	No	Short	Yes	No	No	Black	No	Big	Yes
2.	No	Short	No	No	No	Brown	No	Medium	Yes
3.	Yes	Short	No	No	No	Brown	Yes	Medium	No
4.	No	Short	Yes	No	No	Black	No	Medium	Yes
5.	No	Long	No	Yes	Yes	White	No	Medium	No
6.	No	Short	Yes	Yes	Yes	Black	No	Big	Yes

Generate the set of consistent hypotheses using,

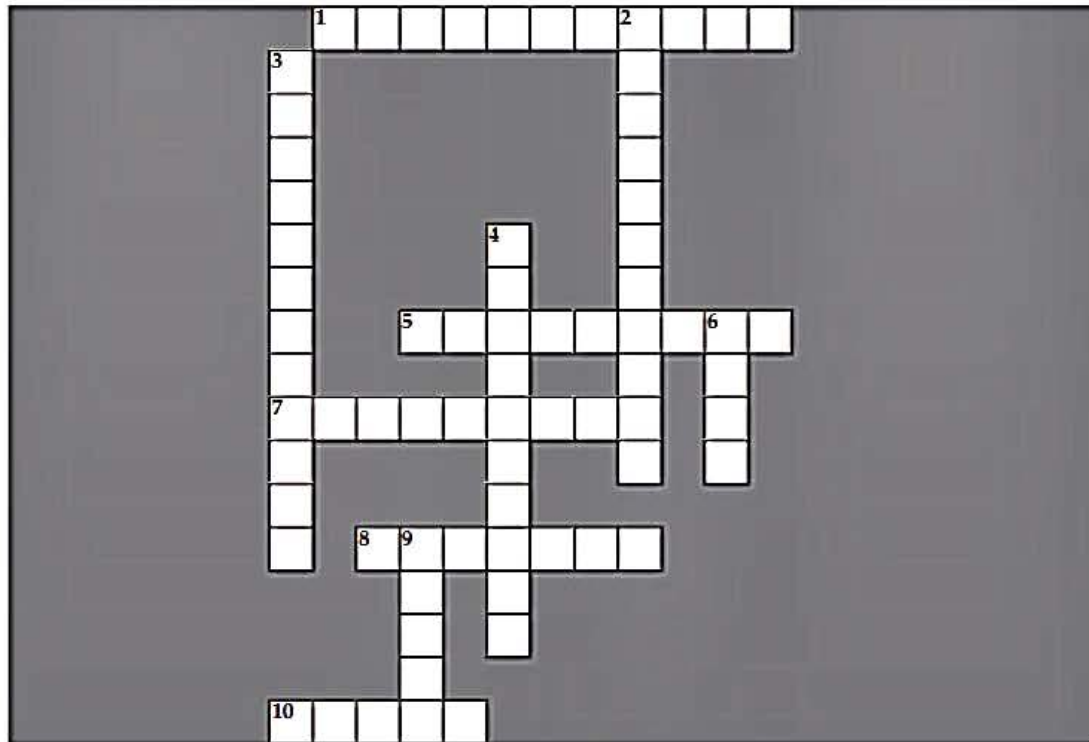
- (a) Find-S algorithm
 - (b) Candidate Elimination algorithm
10. Consider sample training instances shown in Table 3.6, which describe the symptoms of the people and the Covid-19 test result of them. Apply hypothesis search space to generate the set of consistent hypotheses.

Table 3.6: Sample Training Instances

S.No.	Fever	Cough	Throat_Pain	Body_Pain	Covid-19
I1.	N	Y	Y	Y	Positive
I2.	Y	Y	Y	Y	Positive
I3.	Y	N	Y	Y	Positive
I4.	N	Y	N	N	Negative
I5.	Y	Y	Y	N	Positive
I6.	N	N	N	Y	Negative
I7.	N	N	N	N	Negative

11. Define Induction and Inductive bias.
12. Define the three kinds of prediction errors.
13. Discuss the tradeoff between bias and variance.
14. What is meant by overfitting and underfitting?
15. Differentiate between model parameters and hyperparameters.
16. Discuss the different resampling methods.
17. Discuss about the learning frameworks.

Crossword



Across

1. Computational Learning theory deals with _____ aspects of learning theory.
5. Target variable is _____ variable.
7. The learning of target function as an generalization is called _____.
8. The subset of hypothesis from a hypothesis space is called _____ space.
10. MDL uses a _____ for assessing the model.

Down

2. Predictor variables are _____ variables.
3. A learning framework performs _____ analysis of machine learning algorithms.
4. The assumption of the experiment is called _____.
6. The default hypothesis is called _____ hypothesis.
9. The difference between the expected and predicted value is called an _____.

Word Search

Find and mark the words listed below.

H	T	H	E	O	R	E	T	I	C	A	L	S	B	X	N	W	V	N	P	R	F	Y	W	I
S	O	D	A	X	H	H	W	E	A	Y	A	M	H	H	L	G	O	R	V	H	F	W	H	N
L	R	O	Z	V	X	H	W	S	M	I	K	B	J	N	B	I	U	L	J	X	B	V	W	D
R	D	T	J	C	B	M	V	O	A	L	I	W	Q	K	S	F	D	O	N	H	J	D	S	E
K	Q	K	L	L	T	I	A	F	O	E	J	T	N	R	A	W	I	D	O	Y	F	C	W	P
W	P	B	B	Y	G	B	D	T	D	Y	I	U	E	J	N	C	M	B	T	P	X	G	F	E
E	C	B	H	I	A	D	S	D	H	N	T	V	W	N	J	N	E	U	Q	O	I	I	U	N
V	O	X	W	Q	E	X	V	Y	C	E	U	U	D	V	Y	W	B	B	C	T	D	W	C	D
F	K	B	J	P	Z	S	A	V	U	A	M	L	Y	G	T	J	Q	O	U	H	P	D	H	E
F	B	G	D	C	X	W	N	R	I	L	B	A	L	T	Z	T	U	S	E	E	J	R	H	N
B	X	R	X	G	H	J	M	E	R	R	O	R	T	G	M	A	Z	C	Y	S	F	J	F	T
W	M	H	B	D	E	P	E	N	D	E	N	T	Q	I	Z	A	I	O	T	I	J	J	B	S
Z	P	R	Z	M	T	F	T	M	M	F	J	Q	N	X	C	S	E	R	S	S	Z	C	S	P
Q	O	I	N	D	U	C	T	I	O	N	N	C	Z	N	V	A	J	E	P	W	N	X	F	I
T	O	B	A	G	Z	D	Z	S	T	P	A	K	H	C	L	Y	L	A	U	J	D	A	Z	Q
O	H	G	Z	Q	R	O	V	C	O	V	T	D	P	D	G	O	X	P	T	Q	J	I	P	L
D	T	W	O	M	R	Y	I	V	V	K	E	R	G	J	Y	G	S	K	Y	E	R	Q	D	F

Theoretical	Independent	Dependent	Hypothesis	Null	Version
Induction	Error	Mathematical	Score		